INTELLIGENT NOSQL DATABASES

PROJECT REPORT

(Project Semester August-December 2026)

MediTrack AI: Pharmacy Inventory and Demand Forecasting System using Node.js, MongoDB and Python

Submitted by

Tanmay Kumar

Registration No = 12310424

Programme and Section – CSE, K4P23HK

Course Code – CSE494

Under the Guidance of

Dr. Ashu

Discipline of CSE/IT

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara





Table of Contents

1. Problem Statement 3
2. Proposed Solution and Design 3
2.1 Overview of the solution 3
2.2 System architecture 4
2.3 MongoDB data model 4
2.4 Indexing strategy 5
2.5 Main design decisions 5
3. Implementation 6
3.1 Connecting MongoDB and defining schemas 6
3.2 CRUD operations 6
3.3 Selling with FEFO using conditional updates 7
3.4 Aggregation pipelines 7
3.5 Python, PyMongo and demand forecasting 8
3.6 Seeding the database 8
3.7 Application screens 9
3.8 Limitations 10

Project GitHub Link:
https://github.com/tanmaykumar3506/MediTrack-AI

1. Problem Statement

A community pharmacy sells hundreds of different medicines, and every medicine is bought in batches that carry their own expiry date. When stock is kept in notebooks or spreadsheets, it is hard to see which batch is about to expire, which medicine is running low and how much of each medicine will be needed next week. Expired stock is a financial loss and a safety risk for patients, while a medicine that runs out sends the customer away. The pharmacy therefore needs the following.

- **Batch-wise tracking:** one medicine may sit in several batches with different expiry dates, so a single stock number hides the real position.
- **Safe dispensing:** a sale should use only unexpired stock, and the batch that expires first should be sold first (FEFO, first-expiry-first-out).
- **Alerts and insight:** expired batches, batches expiring within 30 days, low-stock medicines, revenue trends and top-selling items must be visible without manual searching.
- **Demand estimation:** the sales history should be used to estimate the demand of each medicine for the next 7 and 30 days, so that reordering is not a guess.

The data of such a pharmacy is semi-structured. A sale contains a variable number of line items, each line points to a medicine and to a batch, and a forecast contains a list of predictions with a list of daily values inside. This kind of nested, evolving data fits a document database better than fixed tables with many joins, which is why **MongoDB is the core of this project**.

The problem addressed in this project can therefore be stated as follows: **to design and build a MongoDB-based pharmacy system that stores medicines, batches, sales, stock adjustments and forecasts as documents, records multi-medicine sales while dispensing the earliest-expiring unexpired batch first, answers inventory and sales questions with indexed queries and aggregation pipelines, and feeds the stored sales history to a Python model that estimates the demand of every medicine for the next 7 or 30 days.**

No real pharmacy data was available, so the project uses generated demo data (204 medicine SKUs and 1,440 sales over 180 days). It is a coursework demonstration and not a validated dispensing system.

2. Proposed Solution and Design

2.1 Overview of the solution

MediTrack AI is a three-part web application. A browser interface sends JSON requests to an Express (Node.js) API, the API reads and writes MongoDB through Mongoose models, and a separate Python program reads the sales from the same database to produce demand forecasts. The frontend uses HTML, CSS, JavaScript and Chart.js; the backend uses Node.js 18+, Express 4, Mongoose 8, express-session, bcryptjs and helmet; the forecasting part uses Python with Pandas, scikit-learn and PyMongo.

2.2 System architecture

Figure 1 shows how the parts are connected. The browser never talks to MongoDB or Python directly. For a forecast request the API starts the Python script as a child process; the script reads the completed sales from MongoDB with PyMongo, and its JSON result is returned to the API, enriched with the current stock and stored in MongoDB as a forecast run.

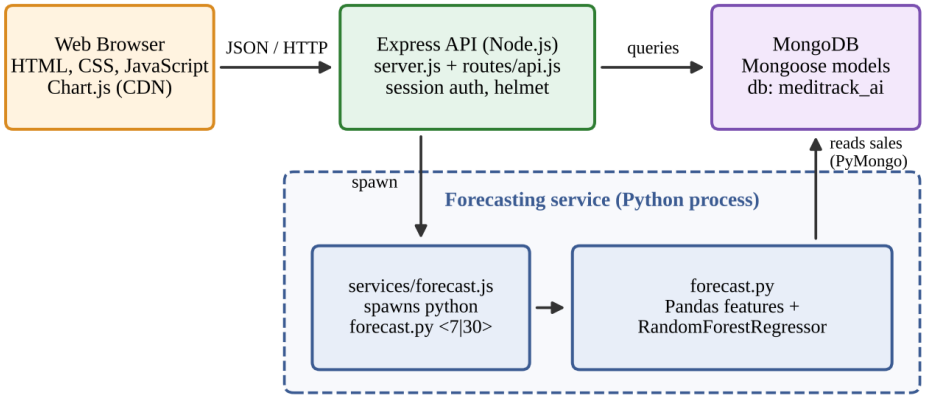


Figure 1: Architecture of MediTrack AI

2.3 MongoDB data model

The database meditrack_ai has five collections (Figure 2): **medicines**, **batches**, **sales**, **stockadjustments** and **forecastruns**. The main modelling question in MongoDB is whether related data should be embedded in one document or kept in another collection and referenced. Table 1 shows the choice made for each relationship and the reason.

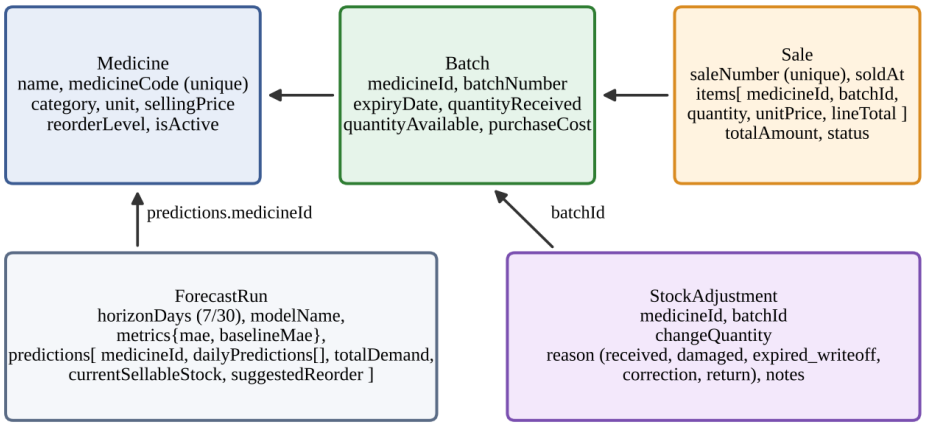


Figure 2: Collections and their references

Table 1: Embedding versus referencing

Relationship	Approach	Reason
Sale and its line items	Embedded array items[]	A sale is always read and written as one unit and has a variable number of lines, so one insert stores it completely.
Item and medicine name, batch number, unit price	Copied into the item	Old sales stay correct even if the catalogue or price changes later.
Batch and medicine	Reference (medicineId)	One medicine has many batches whose quantities change independently as stock is sold.
Sale item and batch	Reference (batchId)	Shows from which batch each unit was sold.
Forecast run and predictions	Embedded arrays (predictions, dailyPredictions)	A run is always displayed as one report, so its parts need no separate queries.
Stock adjustment and batch	Reference (batchId)	The adjustment log grows independently of the batch.

2.4 Indexing strategy

Indexes were defined in the schemas for the queries that the application repeats (Table 2). The unique indexes also protect data quality, because MongoDB rejects a duplicate medicine code, batch number or sale number with error 11000, which the API turns into an HTTP 409 message.

Table 2: Indexes defined in the Mongoose schemas

Collection	Index	Purpose
medicines	text (name, medicineCode); unique medicineCode	Search and duplicate-code prevention
batches	unique (medicineId, batchNumber); medicineId; expiryDate	No duplicate batches; batches of a medicine; expiry ordering for FEFO and alerts
sales	unique saleNumber; soldAt; (items.medicineId, soldAt)	Recent sales; sales history of one medicine
forecastruns	generatedAt	Latest forecast lookup

2.5 Main design decisions

- **FEFO dispensing:** a sale takes stock from the unexpired batch with the earliest expiry date first and may span several batches.
- **Alerts are derived, not stored:** expired, expiring and low-stock conditions are calculated from the current batches at request time, so they are never out of date.
- **Guarded stock updates:** a batch is decremented only if enough quantity is still available, and earlier decrements are restored if a later step fails.
- **Soft delete:** removing a medicine only sets isActive to false, so sales history keeps its references.

- **Forecast kept separate:** inventory and sales pages work without Python; only the forecast page needs it.

3. Implementation

3.1 Connecting MongoDB and defining schemas

The backend connects once at start-up with mongoose.connect, using the MONGODB_URI value from the .env file (a local MongoDB or an Atlas address), and the server starts listening only after the connection succeeds. Every collection is described by a Mongoose schema that adds required fields, minimum values (price and quantity cannot be negative), enumerations (unit, adjustment reason, sale status), trimming and timestamps. Listing 1 shows the Batch schema and the indexes of Table 2. The project is run with npm install, npm run seed and npm start.

Listing 1: Batch schema and indexes (models/index.js, shortened)

```
const BatchSchema = new Schema({
  medicineId:       { type: Schema.Types.ObjectId, ref: 'Medicine', required: true, index: true },
  batchNumber:      { type: String, required: true, trim: true },
  expiryDate:       { type: Date,   required: true },
  quantityReceived:  { type: Number, required: true, min: 0 },
  quantityAvailable: { type: Number, required: true, min: 0 },
  purchaseCost:     { type: Number, required: true, min: 0 }
}, { timestamps: true });
BatchSchema.index({ medicineId: 1, batchNumber: 1 }, { unique: true });
BatchSchema.index({ expiryDate: 1 });
SaleSchema.index({ 'items.medicineId': 1, soldAt: 1 });
```

3.2 CRUD operations

All four CRUD operations are used through Mongoose, as summarised in Table 3. Searching the catalogue uses a case-insensitive $regex filter in which the user text is escaped first, and list pages use lean() so that plain objects are returned.

Table 3: CRUD operations and the Mongoose calls used

CRUD	Mongoose calls	Used for
Create	Medicine.create, Batch.create, Sale.create, StockAdjustment.create, ForecastRun.create, insertMany (seed)	New medicine with first batch, new batch, sale, adjustment, forecast run
Read	find with filter and sort, findOne, findById, populate(medicineId), countDocuments, lean	Catalogue and search, batches by expiry, alerts, dashboard counts
Update	findByIdAndUpdate (runValidators), updateOne with $inc, save	Edit a medicine, decrement stock, stock adjustment
Delete	findByIdAndUpdate with isActive false (soft delete); deleteMany only in the seed script	Archive a medicine without losing history

3.3 Selling with FEFO using conditional updates

A sale may contain several medicines. For each item the route queries the batches that still have stock, sorted by expiryDate, skips expired ones and takes quantity from the earliest batch until the request is covered (Listing 2). If the unexpired stock is not enough, the request is rejected. Each planned decrement is then applied with updateOne whose filter requires quantityAvailable to be at least the quantity taken, together with $inc. MongoDB checks and changes the document in one atomic step, so two simultaneous sales cannot take the same units. If an update matches nothing, the earlier decrements are restored and the API answers with a 409 conflict. The sale document is created only after all decrements succeeded.

Listing 2: FEFO allocation and conditional stock update (api.js, shortened)

```
const batches = await Batch.find({ medicineId: medicine._id,
    quantityAvailable: { $gt: 0 } }).sort({ expiryDate: 1 });
for (const batch of batches) {
  if (!isSellableExpiry(batch.expiryDate)) continue;
  const take = Math.min(remaining, batch.quantityAvailable);
  if (take > 0) { plan.push({ medicine, batch, quantity: take, unitPrice }); remaining -= take; }
  if (!remaining) break;
}
// for every planned line:
const result = await Batch.updateOne(
  { _id: line.batch._id, quantityAvailable: { $gte: line.quantity } },
  { $inc: { quantityAvailable: -line.quantity } });
if (result.modifiedCount !== 1) { /* restore earlier lines, return 409 */ }
```

3.4 Aggregation pipelines

Reports are produced inside MongoDB with aggregation pipelines instead of loading all sales into Node.js (Table 4). The analytics route runs one pipeline with $facet (Listing 3): a single pass over the sales of the chosen period and the period before it returns the totals of both periods, the daily revenue trend, the eight best-selling medicines and the revenue per category. $addFields first computes the units of each sale with $sum over items.quantity, $unwind turns the embedded items into separate documents, and $lookup joins the medicines collection to obtain the category.

Table 4: Aggregation pipelines of the application

Pipeline	Stages	Result
Dashboard revenue (30 days)	$match, $group	Revenue and number of transactions
Daily revenue chart (7 days)	$match, $group with $dateToString, $sort	One revenue value per day
Sales analytics (7/30/90 days)	$match, $addFields, $facet	Current and previous totals, trend, top products, categories
Top products	$unwind, $group, $sort, $limit	Eight best-selling medicines
Category revenue	$unwind, $lookup, $group, $sort	Revenue by medicine category

Listing 3: Aggregation behind the Sales & analytics page (shortened)

```
Sale.aggregate([
  { $match: { status: 'completed', soldAt: { $gte: previousStart } } },
  { $facet: {
   periods:     [ { $group: { _id: { $cond: [{ $gte: ['$soldAt', currentStart] }, 'current', 'previous'] },
                revenue: { $sum: '$totalAmount' }, transactions: { $sum: 1 } } } ],
   topProducts: [ { $unwind: '$items' }, { $group: { _id: '$items.medicineId',
                quantity: { $sum: '$items.quantity' } } }, { $sort: { quantity: -1 } }, { $limit: 8 } ],
   categories:  [ /* $unwind items, $lookup medicines, $group by category */ ] } } ]);
```

3.5 Python, PyMongo and demand forecasting

The forecast uses the same database. When the pharmacist presses the forecast button, the API starts forecast.py for a horizon of 7 or 30 days. The script connects with PyMongo and reads only the fields it needs from the completed sales and the active medicines (Listing 4). It sums the quantity of each medicine per day (including days without sales), creates calendar and lag features, trains a RandomForestRegressor with 120 trees for each medicine, checks it on the last part of the series against a weekly-lag baseline using mean absolute error, and then predicts the future days one by one.

Listing 4: Reading sales from MongoDB in forecast.py

```
client = MongoClient(uri, serverSelectionTimeoutMS=5000)
db = client.get_default_database()
sales = list(db.sales.find({"status": "completed"}, {"soldAt": 1, "items": 1}))
medicines = list(db.medicines.find({"isActive": True}, {"name": 1, "medicineCode": 1}))
```

The API adds the current unexpired stock of each medicine to the result, calculates the suggested reorder as the predicted demand minus that stock, keeps only the medicines where it is positive and stores the run in the forecastruns collection as one document with embedded predictions. The forecast page reads the latest run for the chosen horizon.

3.6 Seeding the database

npm run seed prepares the demo database. It first clears the five collections with deleteMany, then uses insertMany to create 204 medicine SKUs (51 base medicines in four pack sizes), one batch for each of them and 1,440 sales spread over 180 days, with three to six items in each sale. Some batches are deliberately low on stock, close to expiry or already expired, so that every alert type is visible.

3.7 Application screens

The frontend is one HTML page whose JavaScript calls the API and draws the pages with Chart.js. Figures 3 to 5 show the main screens that display data coming from MongoDB.

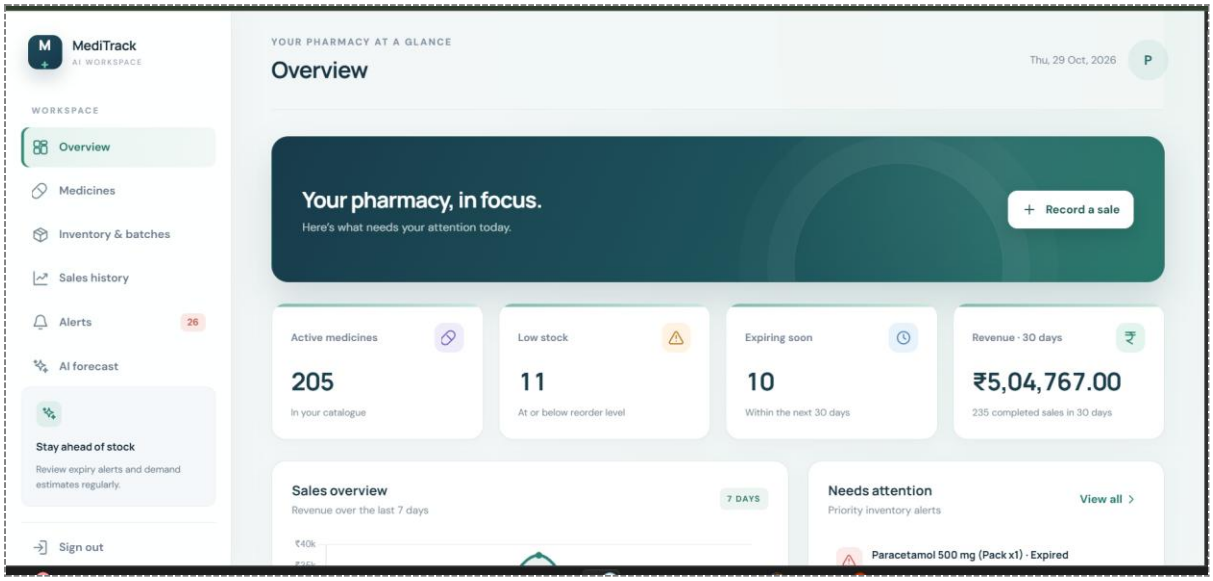


Figure 3: Overview page of MediTrack AI

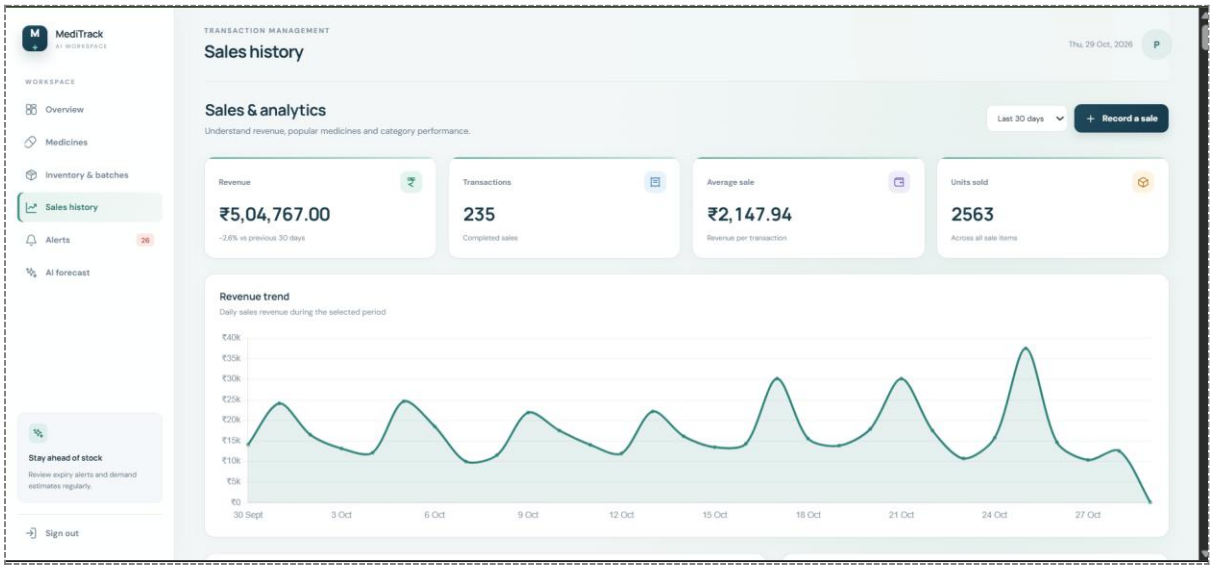


Figure 4: Sales and analytics page built from aggregation pipelines

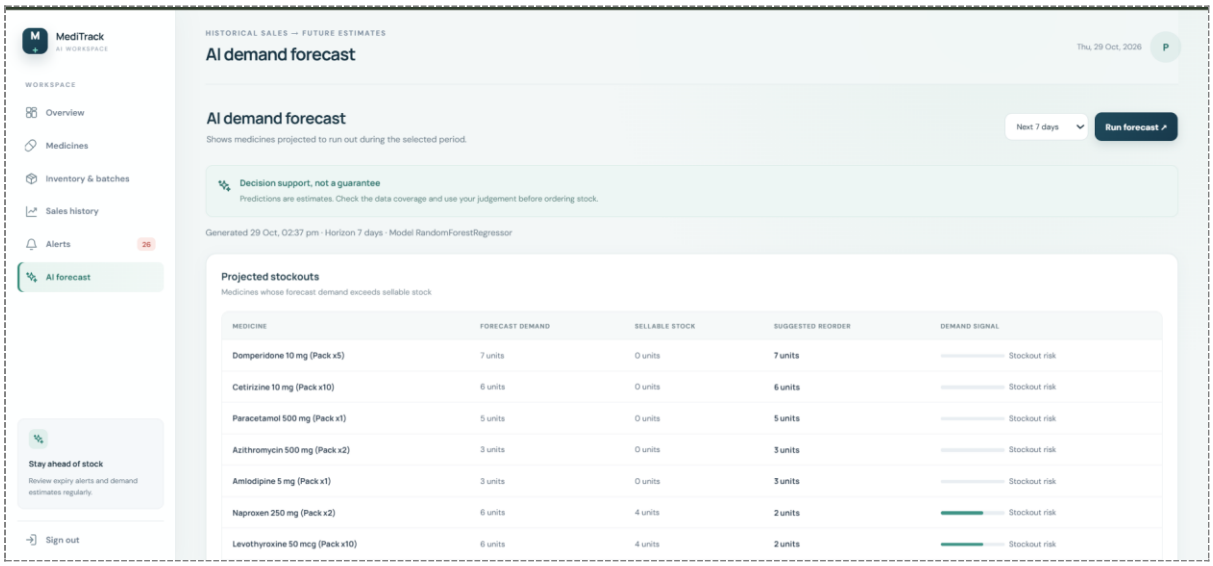


Figure 5: AI demand forecast page

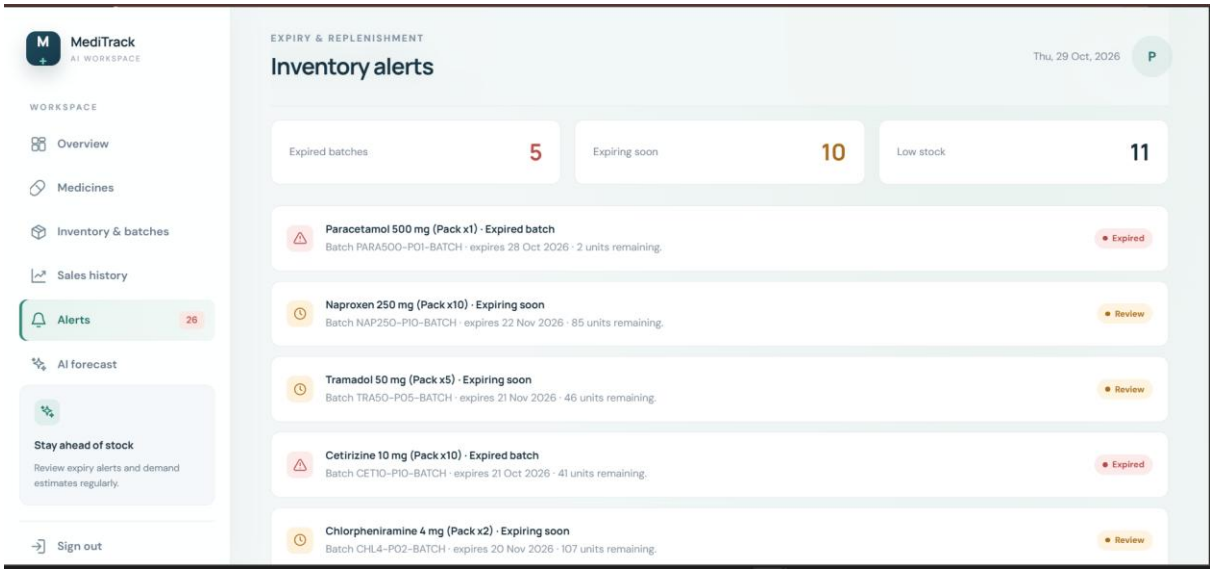


Figure 6: expiry and low stocks alerts

3.8 Limitations

- The data is synthetic, so forecast metrics only show that the pipeline works and say nothing about real pharmacies.
- A sale changes several documents with guarded updates and a manual restore, not with a MongoDB multi-document transaction, so an unlucky failure could still leave inconsistent stock. Transactions need a replica set.
- Login uses one account from .env and the default in-memory session store, which must be replaced before real use.

The complete source code of the project is available at the GitHub repository: **https://github.com/tanmaykumar3506/MediTrack-AI**